



INSTITUT
TEKNOLOGI
HARAPAN
BANGSA
Veritas vos liberabit

MODUL PRAKTIKUM

Teknik Informatika

IF-9P2

Pemrograman Piranti Bergerak

Firhat Hidayat, S.Kom., M.T.

2019 © All rights reserved.

Dilarang memperbanyak dan/atau mengcopy sebagian atau seluruh material
dalam dokumen ini tanpa persetujuan tertulis dari Institut Teknologi Harapan Bangsa

MODUL 8

Background Process

1. Objective

Praktikum ini bertujuan untuk mengimplementasikan *Background Process* pada aplikasi yang Anda bangun. Harapannya aktifitas ini dapat memberi gambaran yang jelas tentang cara kerja proses yang di dalam background.

2. Activities

Implementasi Background Process

3. Exercise

1. Buat Project baru di Android Studio dengan kriteria sebagai berikut:

Nama Project	MyBackgroundThread
Target & Minimum Target SDK	Phone and Tablet, Api level 21
Tipe Activity	Empty Activity
Activity Name	MainActivity
Use AndroidX artifacts	TRUE
Language	Kotlin

2. Jika sudah, kondisikan agar ***activity_main.xml*** menjadi seperti ini:

```
1. <?xml version="1.0" encoding="utf-8"?>
2. <LinearLayout xmlns:android="http://schemas.android.com/apk/res/
   android"
3.     xmlns:tools="http://schemas.android.com/tools"
4.     android:layout_width="match_parent"
5.     android:layout_height="match_parent"
6.     android:gravity="center"
7.     android:orientation="vertical">
8. 
```

```

9.     <TextView
10.         android:id="@+id/tv_status"
11.         android:layout_width="wrap_content"
12.         android:layout_height="wrap_content"
13.         android:text="@string/status" />
14.
15.     <TextView
16.         android:id="@+id/tv_desc"
17.         android:layout_width="wrap_content"
18.         android:layout_height="wrap_content"
19.         tools:text="@string/desc" />
20. </LinearLayout>

```

Kita memiliki sebuah obyek `TextView` dengan ID `tv_status` dan `tv_desc` yang akan menampilkan proses yang sedang dijalankan.

Jangan lupa masukan teks yang ada ke dalam *strings.xml*.

```

1. <resources>
2.     <string name="app_name">MyBackgroundThread</string>
3.     <string name="status">Status Aplikasi</string>
4.     <string name="status_pre">Status : onPreExecute</string>
5.     <string name="status_post">Status : onPostExecute</string>
6.     <string name="desc">Description</string>
7. </resources>

```

3. Selanjutnya kita akan membuat sebuah `interface` dengan nama `MyAsyncCallback` di dalam *MainActivity* seperti berikut:

```

1. class MainActivity : AppCompatActivity(){
2.     override fun onCreate(savedInstanceState: Bundle?) {
3.         super.onCreate(savedInstanceState)
4.         setContentView(R.layout.activity_main)
5.     }
6. }
7.
8. internal interface MyAsyncCallback {
9.     fun onPreExecute()
10.    fun onPostExecute(text: String)
11. }

```

4. Selanjutnya kita masuk ke kelas *MainActivity* untuk menerapkan `MyAsyncCallback` ke kelas tersebut.

```

1. class MainActivity : AppCompatActivity(), MyAsyncCallback {

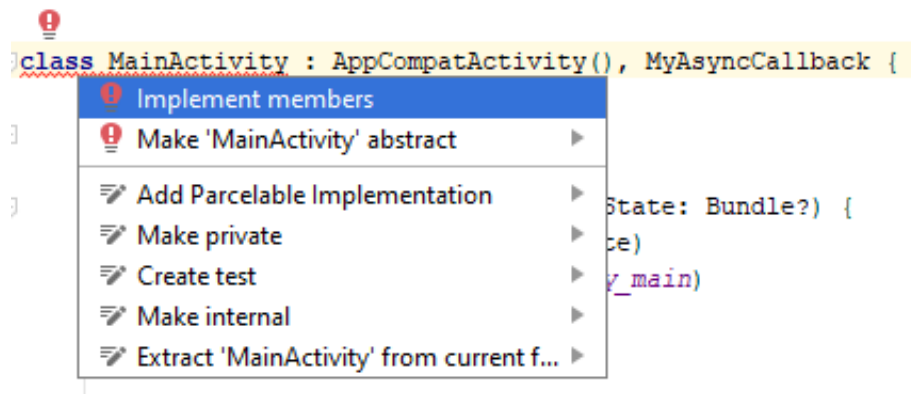
```

```

2.
3.     override fun onCreate(savedInstanceState: Bundle?) {
4.         super.onCreate(savedInstanceState)
5.         setContentView(R.layout.activity_main)
6.
7.     }
8. }
9.
10. internal interface MyAsyncCallback {
11.     fun onPreExecute()
12.     fun onPostExecute(text: String)
13. }

```

5. Selanjutnya akan terdapat garis merah di kelas tersebut, kita harus memperbaikinya dengan cara menekan tombol merah atau **alt+enter**, kemudian pilih **Implement Members (Kotlin)** atau **Implement Methods (Java)** seperti gambar berikut:



Setelah mengimplementasikan kelas **MyAsyncCallbak** ke kelas **MainActivity** maka akan terdapat 2 metode tambahan seperti ini.

```

1. class MainActivity : AppCompatActivity(), MyAsyncCallback {
2.
3.     override fun onCreate(savedInstanceState: Bundle?) {
4.         super.onCreate(savedInstanceState)
5.         setContentView(R.layout.activity_main)
6.
7.     }
8.
9.     override fun onPreExecute() {
10.
11.     }
12.

```

```

13.     override fun onPostExecute(result: String) {
14.         ...
15.     }
16.
17. }
18.
19. internal interface MyAsyncCallback {
20.     fun onPreExecute()
21.     fun onPostExecute(text: String)
22. }

```

6. Selanjutnya kita buat kelas **DemoAsync** yang mana kelas tersebut berfungsi untuk mengelola data secara *asynchronous*.

```

1. class MainActivity : AppCompatActivity(), MyAsyncCallback {
2.     ...
3.     ...
4.     private class DemoAsync(val myListener: MyAsyncCallback) {
5.         companion object {
6.             private val LOG_ASYNC = "DemoAsync"
7.         }
8.         private val myListener: WeakReference<MyAsyncCallback>
9.         init {
10.             this.myListener = WeakReference(myListener)
11.         }
12.     }
13. }
14.
15. }
16. }

```

7. Setelah membuat kelas **DemoAsync** di dalam **MainActivity**, kita akan membuat kelas tersebut mewarisi kelas **AsyncTask** menjadi seperti ini:

```

1. class MainActivity : AppCompatActivity(), MyAsyncCallback {
2.     ...
3.     ...
4.     private class DemoAsync(val myListener: MyAsyncCallback) :
5.         AsyncTask<String, Void, String>() {
6.             companion object {
7.                 private val LOG_ASYNC = "DemoAsync"
8.             }
9.             private val myListener: WeakReference<MyAsyncCallback>
10.

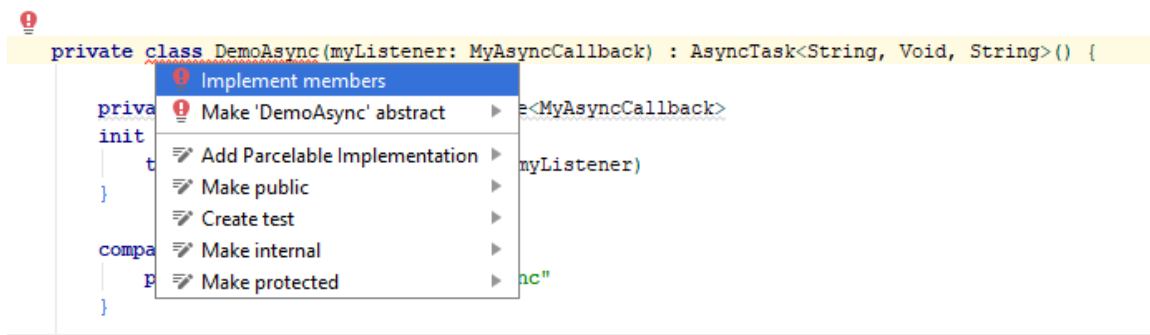
```

```

11.         init {
12.             this.myListener = WeakReference(myListener)
13.         }
14.
15.     }
16. }

```

8. Kemudian kita akan menjumpai garis merah kembali, sama seperti sebelumnya kita akan memperbaikinya dengan menekan tombol merah atau **alt+enter** seperti gambar berikut:



9. Sehingga kode ke dalam kelas **DemoAsync** menjadi seperti ini:

```

1. private class DemoAsync(val myListener: MyAsyncCallback) :
   AsyncTask<String, Void, String>() {
2.     ...
3.
4.     override fun doInBackground(vararg p0: String?): String {
5.         TODO("not implemented") //To change body of created functions
        use File | Settings | File Templates.
6.     }

```

10. Setelah itu, Anda bisa menambahkan metode-metode di dalam kelas **DemoAsync** untuk persiapan dan ketika proses sudah berhasil.

```

1. private class DemoAsync(val myListener: MyAsyncCallback) :
   AsyncTask<String, Void, String>() {
2.
3.     ...
4.
5.     override fun onPreExecute() {
6.         super.onPreExecute()
7.         Log.d(LOG_ASYNC, "status : onPreExecute")
8.
9.         val myListener = myListener.get()
10.        myListener?.onPreExecute()
11.    }
12.
13.    override fun doInBackground(vararg params: String?): String {
14.        Log.d(LOG_ASYNC, "status : doInBackground")
15.
16.        var output: String? = null
17.
18.        try {
19.            val input = params[0]
20.            output = "$input Selamat Belajar!!"
21.            Thread.sleep(2000)
22.        } catch (e: Exception) {
23.            Log.d(LOG_ASYNC, e.message)
24.        }
25.
26.        return output.toString()
27.    }
28.
29.    override fun onPostExecute(result: String) {
30.        super.onPostExecute(result)
31.        Log.d(LOG_ASYNC, "status : onPostExecute")
32.
33.        val myListener = this.myListener.get()
34.        myListener?.onPostExecute(result)
35.    }
36. }

```

Maka kelas **DemoAsync** menjadi seperti berikut ini:

```

1. private class DemoAsync(val myListener: MyAsyncCallback) :
   AsyncTask<String, Void, String>() {
2.
3.     companion object {
4.         private val LOG_ASYNC = "DemoAsync"
5.     }
6.     private val myListener: WeakReference<MyAsyncCallback>
7.     init {
8.         this.myListener = WeakReference(myListener)
9.     }
10.
11.     override fun onPreExecute() {
12.         super.onPreExecute()
13.         Log.d(LOG_ASYNC, "status : onPreExecute")
14.
15.         val myListener = myListener.get()
16.         myListener?.onPreExecute()
17.     }
18.
19.     override fun doInBackground(vararg params: String?): String {
20.         Log.d(LOG_ASYNC, "status : doInBackground")
21.
22.         var output: String? = null
23.
24.         try {
25.             val input = params[0]
26.             output = "$input Selamat Belajar!!"
27.             Thread.sleep(2000)
28.         } catch (e: Exception) {
29.             Log.d(LOG_ASYNC, e.message)
30.         }
31.
32.         return output.toString()
33.     }
34.
35.     override fun onPostExecute(result: String) {
36.         super.onPostExecute(result)
37.         Log.d(LOG_ASYNC, "status : onPostExecute")
38.
39.         val myListener = this.myListener.get()
40.         myListener?.onPostExecute(result)
41.     }
42. }

```

11. Selanjutnya, Anda perlu menambahkan kode berikut untuk menjalankan proses AsyncTask di kelas **MainActivity**.


```

1. class MainActivityUsingAsyncTask : AppCompatActivity(), MyAsyncCallback
2. {
3.     companion object {
4.         private const val INPUT_STRING = "Halo Ini Demo AsyncTask!!"
5.     }
6.
7.     override fun onCreate(savedInstanceState: Bundle?) {
8.         super.onCreate(savedInstanceState)
9.         setContentView(R.layout.activity_main)
10.
11.         val demoAsync = DemoAsync(this)
12.         demoAsync.execute(INPUT_STRING)
13.     }

```

Jangan lupa memberikan aksi ketika proses berlangsung di bagian *callback* dari kelas **MyAsyncCallback**.

```

1. class MainActivity : AppCompatActivity(), MyAsyncCallback {
2.     ...
3.
4.     override fun onPreExecute() {
5.         tv_status.setText(R.string.status_pre)
6.         tv_desc.text = INPUT_STRING
7.     }
8.
9.     override fun onPostExecute(result: String) {
10.        tv_status.setText(R.string.status_post)
11.        tv_desc.text = result
12.    }
13.    ...
14. }

```

Sehingga kelas **MainActivity** menjadi seperti berikut:

```

1. class MainActivity : AppCompatActivity(), MyAsyncCallback {
2.
3.     companion object {
4.         private const val INPUT_STRING = "Halo Ini Demo AsyncTask!!"
5.     }
6.
7.     override fun onCreate(savedInstanceState: Bundle?) {
8.         super.onCreate(savedInstanceState)
9.         setContentView(R.layout.activity_main)
10.
11.         val demoAsync = DemoAsync(this)

```

```

12.     demoAsync.execute(INPUT_STRING)
13. }
14.
15.
16.     override fun onPreExecute() {
17.         tv_status.setText(R.string.status_pre)
18.         tv_desc.text = INPUT_STRING
19.     }
20.
21.     override fun onPostExecute(result: String) {
22.         tv_status.setText(R.string.status_post)
23.         tv_desc.text = result
24.     }
25.
26.     private class DemoAsync(myListener: MyAsyncCallback) :
        AsyncTask<String, Void, String>() {
27.
28.         private val myListener: WeakReference<MyAsyncCallback>
29.         init {
30.             this.myListener = WeakReference(myListener)
31.         }
32.
33.         companion object {
34.             private val LOG_ASYNC = "DemoAsync"
35.         }
36.
37.         override fun onPreExecute() {
38.             super.onPreExecute()
39.             Log.d(LOG_ASYNC, "status : onPreExecute")
40.
41.             val myListener = this.myListener.get()
42.             myListener?.onPreExecute()
43.         }
44.
45.         override fun doInBackground(vararg params: String?): String {
46.             Log.d(LOG_ASYNC, "status : doInBackground")
47.
48.             var output: String? = null
49.
50.             try {
51.                 val input = params[0]
52.                 output = "$input Selamat Belajar!!"
53.                 Thread.sleep(2000)
54.
55.             } catch (e: Exception) {
56.                 Log.d(LOG_ASYNC, e.message)
57.             }
58.
59.             return output.toString()
60.         }

```

```

61.
62.     override fun onPostExecute(result: String) {
63.         super.onPostExecute(result)
64.         Log.d(LOG_ASYNC, "status : onPostExecute")
65.
66.         val myListener = this.myListener.get()
67.         myListener?.onPostExecute(result)
68.     }
69. }
70. }
71.
72. internal interface MyAsyncCallback {
73.     fun onPreExecute()
74.     fun onPostExecute(text: String)
75. }

```

12. Sekarang, silakan jalankan aplikasi **MyAsyncTask** dan perhatikan obyek **TextView** akan melakukan perubahan status terhadap obyek **DemoAsyncTask** yang dijalankan.